

Authorization (नेपालीमा अनुमति) भनेको कुन प्रयोगकर्ताले एपमा के के गर्न पाउँछ र के के गर्न पाउँदैन भनेर तोक्ने प्रक्रिया हो। [\[1\]](#), [\[2\]](#)

Authentication (जस्तै: Login गर्ने प्रक्रिया) ले तपाईं **को हो** भनेर चिनाउँछ भने, Authorization ले तपाईंले **के गर्न पाउनुहुन्छ** भन्ने कुराको नियन्त्रण गर्छ। [\[1\]](#)

यसलाई बुझ्न सजिलो हुने गरी Node.js/Express र Database को कन्सेप्ट प्रयोग गरेर बुँदागत रूपमा विस्तृतमा व्याख्या गरिएको छ:

1. Role-Based Access Control (RBAC)

यो एउटा यस्तो प्रणाली हो जहाँ हामी सिधै एउटा-एउटा प्रयोगकर्तालाई अनुमति दिनुको सट्टा "Roles" (भूमिकाहरू) बनाउँछौं। त्यसपछि ती भूमिकाहरू प्रयोगकर्तालाई दिइन्छ। [\[1\]](#), [\[2\]](#)

- **फाइदा:** यदि १००० जना युजर्स छन् भने सबैलाई छुट्टाछुट्टै अनुमति मिलाइरहनु पर्दैन। रोल परिवर्तन गर्नासाथ त्यो रोल भएका सबै युजर्सको अनुमति एकैपटक परिवर्तन हुन्छ।
 - **उदाहरण:** कलेजको एप्लिकेसनमा `ADMIN`, `TEACHER`, र `STUDENT` अलग-अलग रोलहरू हुन्। [\[1\]](#), [\[2\]](#)
-

2. Admin vs User

रोल बेस सिस्टममा यो सबैभन्दा सामान्य वर्गीकरण हो:

- **Admin (एडमिन):** एपको सुपरपावर भएको अकाउन्ट हो। यसले डाटाहरू Create, Read, Update, र Delete (CRUD) सबै गर्न सक्छ। यसले अरू युजर थप्न वा हटाउन पनि सक्छ। [\[1\]](#), [\[2\]](#), [\[3\]](#)
 - **User (सामान्य प्रयोगकर्ता):** सीमित अधिकार भएको अकाउन्ट हो। यसले केवल आफ्नै प्रोफाइल हेर्ने, सम्पादन गर्ने वा उसलाई तोकिएको साधारण कामहरू मात्र गर्न पाउँछ। [\[1\]](#), [\[2\]](#)
-

3. Permissions (अनुमतिहरू)

पर्मिसन भनेको कुनै पनि रिसोर्समा गर्न पाइने **विशिष्ट एक्सन (Specific Action)** हो। रोलहरू सधैं पर्मिसनहरूको समूहबाट बन्छन्। [\[1, 2, 3\]](#)

- **कन्सेप्ट:** Role = Group of Permissions [\[1, 2\]](#)
 - **उदाहरण:**
 - create:product (सामान थप्ने अनुमति)
 - delete:blog (ब्लग हटाउने अनुमति) [\[1, 2\]](#)
 - **म्यापिङ:** Admin रोलसँग ['create:product', 'delete:blog', 'view:analytics'] पर्मिसन हुन सक्छ। User रोलसँग केवल ['view:product'] मात्र हुन सक्छ। [\[1, 2\]](#)
-

4. Resource Ownership (श्रोतको स्वामित्व)

कतिपय अवस्थामा रोल User भए पनि उसले आफ्नै डेटा मात्र चलाउन पाउनुपर्छ, अरूको पाउनुहुँदैन। यसैलाई **Resource Ownership** भनिन्छ। [\[1, 2, 3, 4\]](#)

- **समस्या:** User रोल भएको राम र श्याम दुवैजना छन्। रामले आफ्नो ब्लग पोस्ट Update गर्न पाउनु पर्छ, तर उसले श्यामको ब्लग पोस्ट Update गर्न पाउनुहुँदैन।
- **समाधान:** कोड लेख्दा हामी डेटाबेसको userId र लगइन भएको युजरको id एउटै छ कि छैन भनेर चेक गर्छौं:

javascript

```
if (post.authorId !== req.user.id) {  
  return res.status(403).json({ message: "You don't own this  
resource" });  
}
```

Use code with caution.

[\[1\]](#)

5. Route Protection (राउट संरक्षण)

प्रयोगकर्ताले सिधै URL वा API Endpoint (जस्तै: /admin/dashboard) हानेर अनधिकृत डेटा नहेरून् भन्नाका लागि Middleware मार्फत राउट लक गर्ने प्रक्रिया नै राउट प्रोटेक्सन हो. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#), [\[5\]](#)

- **काम गर्ने तरिका:** जब कुनै युजरले कुनै राउट रिक्वेस्ट गर्छ, ब्याकइन्डमा एउटा **Authorization Middleware** चल्छ। यसले युजरको टोकन (JWT) वा सेसन चेक गर्छ। [\[1\]](#), [\[2\]](#), [\[3\]](#)

व्यावहारिक कोडको उदाहरण (Node.js/Express):

javascript

```
// १. रोल चेक गर्ने Middleware
const authorizeRoles = (...allowedRoles) => {
  return (req, res, next) => {
    // req.user Authentication middleware बाट आउँछ
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: "यो ठाउँमा जान तपाईंलाई अनुमति छैन!" });
    }
    next(); // अनुमति छ भने अगाडि बढ्ने
  };
};

// २. Route Protection को प्रयोग
// यो राउटमा केवल Admin र Manager मात्र छिर्न पाउँछन्
app.delete('/api/v1/products/:id', authenticateUser,
authorizeRoles('ADMIN', 'MANAGER'), deleteProduct);
Use code with caution.
```

यस प्रक्रियालाई अझ प्रष्ट पार्न **डाटाबेस तालिका (Database Schema)** कसरी बनाइन्छ, तलको तालिकाबाट बुझ्न सकिन्छ: [\[1\]](#)

Table (तालिका)	Fields (फिल्डहरू)	विवरण
Users	id, name, email, password	प्रयोगकर्ताको विवरण
Roles	id, role_name (ADMIN, USER)	रोलहरूको सूची
User_Roles	user_id, role_id	कुन युजरको के रोल हो भनेर जोड्ने (Many-to-Many)